

Culture et fondations DevOps/SRE

Maxime El Haddari

September 16, 2026

1 Introduction

- L'objectif du cours
- Méthodes d'évaluation
- Le programme

2 C'est quoi le DevOps ?

- Historique
- La culture SRE
 - La culture SRE - Exemples

3 Infrastructure As Code

- Généralités
- Terraform

4 Software Delivery Lifecycle

- Comment développer ?
 - Trunk-based Development
 - Comment review ?
 - Utiliser les Feature Flags
 - Les conventions de commits
 - lint + pre-commit



Maxime El Haddari
Senior SRE @Sekoia

linkedin.com/maxime-el-haddari
maxime.elhaddari@gmail.com
github.com/maxime-elhaddari

Ce que vous allez apprendre:

- Comment faire de l'Infrastructure à grande échelle

Ce que vous allez apprendre:

- Comment faire de l'Infrastructure à grande échelle
- Comment opérer du code en production

Ce que vous allez apprendre:

- Comment faire de l'Infrastructure à grande échelle
- Comment opérer du code en production
- Comprendre les enjeux d'une équipe Infrastructure

Comment vous allez apprendre:

- Je vous partage mes connaissances

Comment vous allez apprendre:

- Je vous partage mes connaissances
- Vos questions

Comment vous allez apprendre:

- Je vous partage mes connaissances
- Vos questions
- Les TP pour expérimenter réellement

Comment vous allez être évalué :

- Passage à l'oral rapide : quelques questions en 5 minutes

Comment vous allez être évalué :

- Passage à l'oral rapide : quelques questions en 5 minutes
- Les TP via git : il faudra me donner accès à votre projet et les commits seront pris en compte

Comment vous allez être évalué :

- Passage à l'oral rapide : quelques questions en 5 minutes
- Les TP via git : il faudra me donner accès à votre projet et les commits seront pris en compte
- Participations spontanées : side project, exposés, etc...

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?
- Jour 2 : Comment j'envoie mon code en production ?

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?
- Jour 2 : Comment j'envoie mon code en production ?
- Jour 3 : Comment on peut scale son code et la production ?

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?
- Jour 2 : Comment j'envoie mon code en production ?
- Jour 3 : Comment on peut scale son code et la production ?
- Jour 4 : Comment on gère la production ?

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?
- Jour 2 : Comment j'envoie mon code en production ?
- Jour 3 : Comment on peut scale son code et la production ?
- Jour 4 : Comment on gère la production ?
- Jour 5 : Comment on sécurise la production ?

- Jour 1 : C'est quoi le DevOps ? Et le SRE dans tout ça ?
- Jour 2 : Comment j'envoie mon code en production ?
- Jour 3 : Comment on peut scale son code et la production ?
- Jour 4 : Comment on gère la production ?
- Jour 5 : Comment on sécurise la production ?
- Jour 6 : Une Data Platform, c'est quoi ?

- C'est quoi le DevOps selon vous ? Et le SRE ?

- C'est quoi le DevOps selon vous ? Et le SRE ?
- D'après vous quels sont les enjeux qui requièrent un DevOps ? Un SRE ?

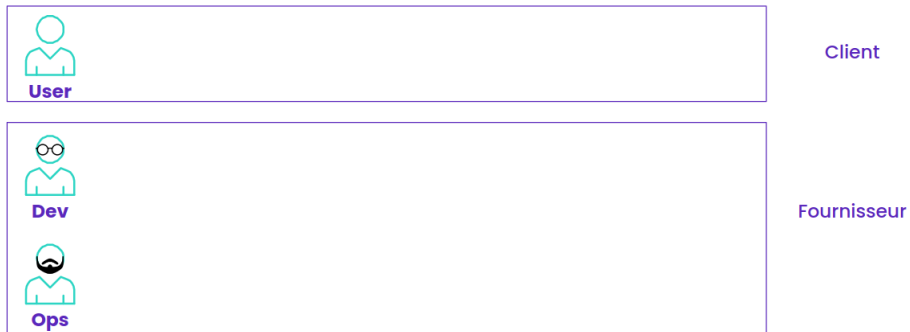
- C'est quoi le DevOps selon vous ? Et le SRE ?
- D'après vous quels sont les enjeux qui requièrent un DevOps ? Un SRE ?
- Quels sont leurs challenges et leurs enjeux ?

Le **DevOps** : une culture et une façon de travailler pour améliorer la vitesse, la fiabilité, la sécurité et le coût d'un projet de développement informatique.

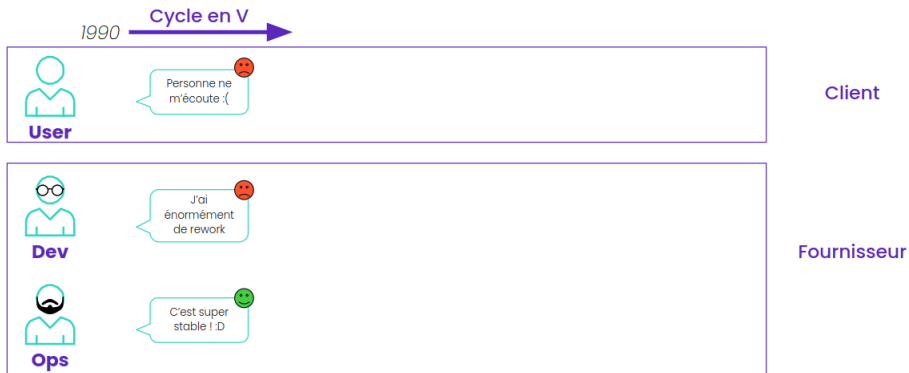
Le principe principal consiste à réconcilier le développement et la partie opération/production.



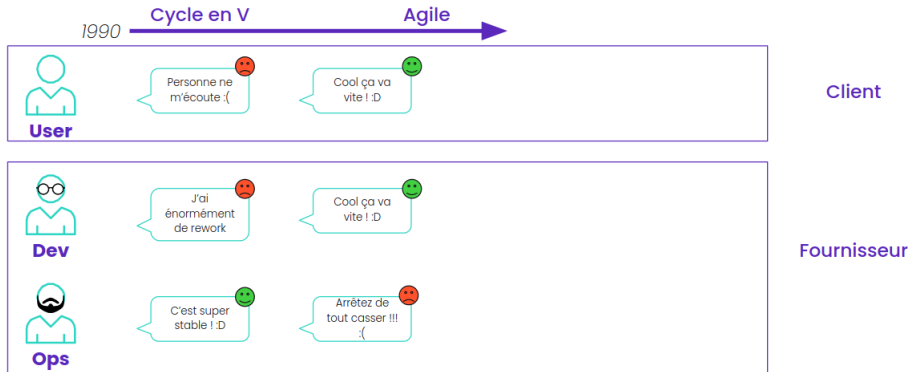
L'évolution du trio User / Dev / Ops



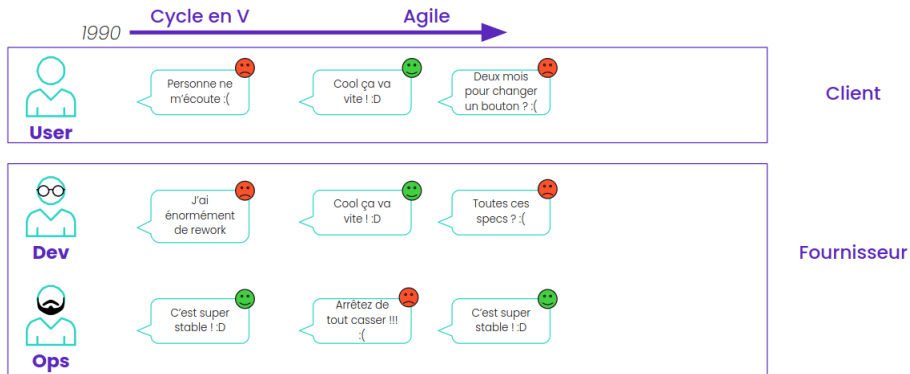
L'évolution du trio User / Dev / Ops



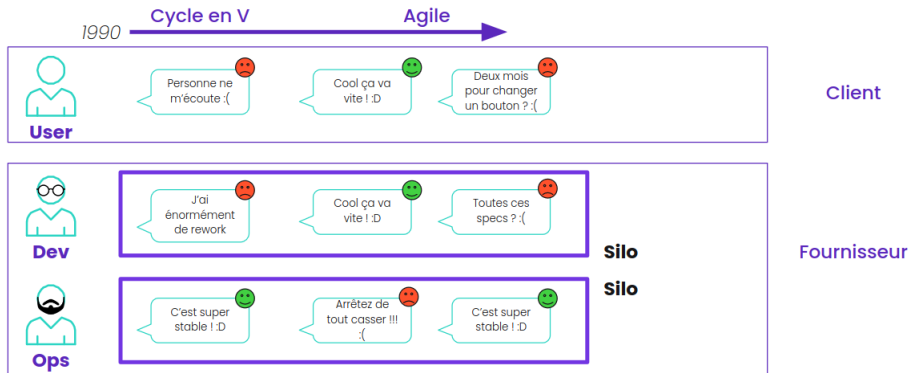
L'évolution du trio User / Dev / Ops



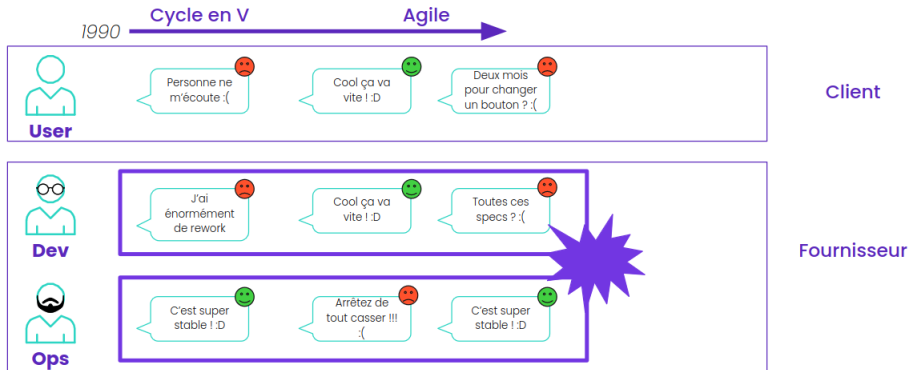
L'évolution du trio User / Dev / Ops



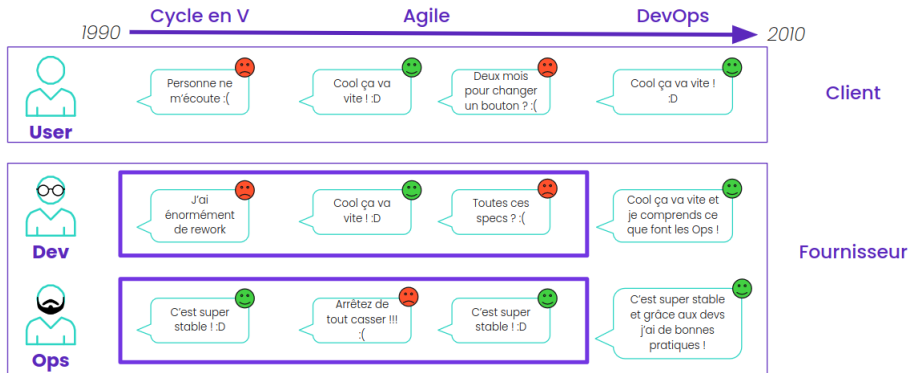
L'évolution du trio User / Dev / Ops



L'évolution du trio User / Dev / Ops



L'évolution du trio User / Dev / Ops



Un DevOps est là pour :

- Assurer que la chaîne de développement et la livraison soit efficaces
- Automatiser les process afin d'éliminer les frictions entre Dev et Ops
- Empêcher les silos en intégrant les Dev aux Ops et les Ops aux Dev
- Assurer une livraison continue à l'aide de compétences transversales

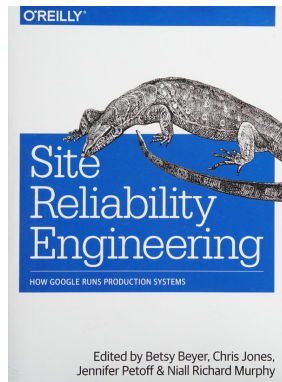
Et le SRE dans tout ça ?

En 2016, Google sort son livre **Site Reliability Engineering**.

Ils expliquent leur vision du DevOps et leurs méthodes.

Les principaux concepts développés sont :

- SLI / SLO / SLA
- Error budget
- Toil
- Incident management



<https://sre.google/books/>

Le **SRE : Site Reliability Engineer** est un métier, mais surtout une pratique d'ingénierie pour atteindre et maintenir la fiabilité ainsi que la qualité et la performance des services. C'est une application pratique de la culture DevOps.

'SRE is what happens when you ask a software engineer to design an operations team.' Ben Treynor - SRE Founder

Le SRE se base sur :

- Des mesures chiffrées

Le SRE se base sur :

- Des mesures chiffrées
- Un suivi permanent de ces mesures : SLI / SLO et SLA + budget d'erreur

Le SRE se base sur :

- Des mesures chiffrées
- Un suivi permanent de ces mesures : SLI / SLO et SLA + budget d'erreur
- Beaucoup d'automatisations afin d'assurer des opérations efficaces et fluides

Le SRE se base sur :

- Des mesures chiffrées
- Un suivi permanent de ces mesures : SLI / SLO et SLA + budget d'erreur
- Beaucoup d'automatisations afin d'assurer des opérations efficaces et fluides
- Un système qui échoue avec grâce

En tant que **SRE**, je dois assurer:

- la disponibilité 24/7 worldwide

En tant que **SRE**, je dois assurer:

- la disponibilité 24/7 worldwide
- la scalabilité de la production

En tant que **SRE**, je dois assurer:

- la disponibilité 24/7 worldwide
- la scalabilité de la production
- le temps minimal de livraison applicative (time from code to user)

En tant que **SRE**, je dois assurer:

- la disponibilité 24/7 worldwide
- la scalabilité de la production
- le temps minimal de livraison applicative (time from code to user)
- la sécurité par design

En tant que **SRE**, je dois assurer:

- la disponibilité 24/7 worldwide
- la scalabilité de la production
- le temps minimal de livraison applicative (time from code to user)
- la sécurité par design
- la visibilité sur la vie de l'application (métriques et erreurs)

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** $\Rightarrow$ #AutomationFirst

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** $\Rightarrow$ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** $\Rightarrow$ #FailSafe

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** $\Rightarrow$ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** $\Rightarrow$ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** $\Rightarrow$ #ThinkBig

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** $\Rightarrow$ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** $\Rightarrow$ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** $\Rightarrow$ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** $\Rightarrow$ #Observability

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** $\Rightarrow$ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** $\Rightarrow$ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** $\Rightarrow$ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** $\Rightarrow$ #Observability
- **Sécurité intégrée, pas à la fin si on a le temps** $\Rightarrow$ #Shiftleft

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** ⇒ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** ⇒ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** ⇒ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** ⇒ #Observability
- **Sécurité intégrée, pas à la fin si on a le temps** ⇒ #Shiftleft
- **Partage ton savoir comme tu partages tes memes** ⇒ #Teamplay

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** ⇒ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** ⇒ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** ⇒ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** ⇒ #Observability
- **Sécurité intégrée, pas à la fin si on a le temps** ⇒ #Shiftleft
- **Partage ton savoir comme tu partages tes memes** ⇒ #Teamplay
- **Pas de héros de prod, juste des équipes qui dorment la nuit** ⇒ #Reliability

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** ⇒ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** ⇒ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** ⇒ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** ⇒ #Observability
- **Sécurité intégrée, pas à la fin si on a le temps** ⇒ #Shiftleft
- **Partage ton savoir comme tu partages tes memes** ⇒ #Teamplay
- **Pas de héros de prod, juste des équipes qui dorment la nuit** ⇒ #Reliability
- **Le script qui dort sur ton /home n'est pas ton ami** ⇒ #InfraAsCode

Les valeurs du **SRE**:

- **Ne jamais faire 2 fois la même action** ⇒ #AutomationFirst
- **Toujours casser en staging avant de casser en prod** ⇒ #FailSafe
- **Si ça ne scale pas, c'est que ça ne vaut pas le coup** ⇒ #ThinkBig
- **Toujours avoir des métriques, même pour ta bouilloire** ⇒ #Observability
- **Sécurité intégrée, pas à la fin si on a le temps** ⇒ #Shiftleft
- **Partage ton savoir comme tu partages tes memes** ⇒ #Teamplay
- **Pas de héros de prod, juste des équipes qui dorment la nuit** ⇒ #Reliability
- **Le script qui dort sur ton /home n'est pas ton ami** ⇒ #InfraAsCode
- **Tu n'es pas ton job, mais tu es l'âme du pipeline** ⇒ #DevOps

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du
site ou du produit ou de la
plateforme ou de l'application

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du site ou du produit ou de la plateforme ou de l'application
- SLO - Service Level Objective :
Objectif de disponibilité fixé par l'équipe sur une période donnée, mesuré par les SLIs

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du site ou du produit ou de la plateforme ou de l'application
- SLO - Service Level Objective :
Objectif de disponibilité fixé par l'équipe sur une période donnée, mesuré par les SLIs
- SLA - Service Level Agreement :
Engagement pris par l'équipe auprès des utilisateurs, supérieurs aux SLOs

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du site ou du produit ou de la plateforme ou de l'application
- SLO - Service Level Objective :
Objectif de disponibilité fixé par l'équipe sur une période donnée, mesuré par les SLIs
- SLA - Service Level Agreement :
Engagement pris par l'équipe auprès des utilisateurs, supérieurs aux SLOs
- Error budget :
 $100 - \text{SLA} = \text{marge de manœuvre restante}$

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du site ou du produit ou de la plateforme ou de l'application
- SLO - Service Level Objective :
Objectif de disponibilité fixé par l'équipe sur une période donnée, mesuré par les SLIs
- SLA - Service Level Agreement :
Engagement pris par l'équipe auprès des utilisateurs, supérieurs aux SLOs
- Error budget :
 $100 - \text{SLA} = \text{marge de manœuvre restante}$
- Toil :
Tâches manuelles, répétitives et automatisables (sans plus-value)

- SLI - Service Level Indicator :
Indicateurs de la disponibilité du site ou du produit ou de la plateforme ou de l'application
- SLO - Service Level Objective :
Objectif de disponibilité fixé par l'équipe sur une période donnée, mesuré par les SLIs
- SLA - Service Level Agreement :
Engagement pris par l'équipe auprès des utilisateurs, supérieurs aux SLOs
- Error budget :
 $100 - \text{SLA} = \text{marge de manœuvre restante}$
- Toil :
Tâches manuelles, répétitives et automatisables (sans plus-value)
- Incident management :
Traçabilité des incidents afin d'apprendre et de les éviter = post mortem

SLI / SLO / SLA :

- 99.9% SLA sur 30 jours = 43,2 minutes d'"*error budget*"

SLI / SLO / SLA :

- 99.9% SLA sur 30 jours = 43,2 minutes d'"*error budget*"
- 20 min d'indisponibilité = 46% du "*error budget*" consommé donc on ralentit les déploiements tant que le "*error budget*" n'est pas reconstitué

SLI / SLO / SLA :

- 99.9% SLA sur 30 jours = 43,2 minutes d'"*error budget*"
- 20 min d'indisponibilité = 46% du "*error budget*" consommé donc on ralentit les déploiements tant que le "*error budget*" n'est pas reconstitué
- 100% ou plus du "*error budget*" consommé = on bloque toutes les releases de feature

SLI / SLO / SLA :

- 99.9% SLA sur 30 jours = 43,2 minutes d'"*error budget*"
- 20 min d'indisponibilité = 46% du "*error budget*" consommé donc on ralentit les déploiements tant que le "*error budget*" n'est pas reconstitué
- 100% ou plus du "*error budget*" consommé = on bloque toutes les releases de feature
- Burn-rate = vitesse à laquelle on crame son "*error budget*"
Par exemple:
1,44 minutes d'"*error budget*" par jour $\Leftrightarrow$ *burn-rate* = 1

Incident management :

- Système d'alerte basé sur les métriques, si l'alerte est P1 ou plus

Incident management :

- Système d'alerte basé sur les métriques, si l'alerte est P1 ou plus
- On ouvre un canal dédié et on réunit les personnes pertinentes

Incident management :

- Système d'alerte basé sur les métriques, si l'alerte est P1 ou plus
- On ouvre un canal dédié et on réunit les personnes pertinentes
- On attribue des rôles : Commander, Communicator, Operator

Incident management :

- Système d'alerte basé sur les métriques, si l'alerte est P1 ou plus
- On ouvre un canal dédié et on réunit les personnes pertinentes
- On attribue des rôles : Commander, Communicator, Operator
- On note les actions effectuées et on communique

Incident management :

- Système d'alerte basé sur les métriques, si l'alerte est P1 ou plus
- On ouvre un canal dédié et on réunit les personnes pertinentes
- On attribue des rôles : Commander, Communicator, Operator
- On note les actions effectuées et on communique
- À la fin, on ouvre un post mortem

Incident management :

Preliminary Post Incident Review (PIR): Content Configuration Update Impacting the Falcon Sensor and the Windows Operating System (BSOD)

July 24, 2024 • CrowdStrike • Executive Viewpoint



Updated 2024-07-25 1900 UTC

[Executive Summary PDF](#)

This is CrowdStrike's preliminary Post Incident Review (PIR). We will be detailing our full investigation in the forthcoming Root Cause Analysis that will be released publicly. Throughout this PIR, we have used generalized terminology to describe the Falcon platform for improved readability. Terminology in other documentation may be more specific and technical.

Traçabilité des incidents :

- Communication interne ou externe (coup de pub)
- Root Cause Analysis (RCA)
- Apprendre de ses erreurs

Exemple de post mortem

Incident management :

Postmortem Template

This is a standard template we use for postmortems at PagerDuty — feel free to use it for your own!

Guidelines This page is intended to be reviewed during a postmortem meeting that should be scheduled within 5 business days of any event. Your first step should be to schedule the postmortem meeting for within 5 business days after the incident. Don't wait until you've filed in the site to schedule the meeting. However, make sure this page is completed by the meeting.	Postmortem Owner: Your name goes here. Meeting Scheduled for: Schedule the meeting for within 5 business days after the incident. Put the date/time here. Call Recording: Link to the incident call recording.
Overview: Include a short sentence or two summarizing the root causes, timeline summary, and the impact. (e.g. "On the morning of August 19th, we suffered a service Degrade due to a software process on our primary database machines. This process caused roughly 500k% of data that had begun during this time to be delivered out of SLA.")	
What Happened: Include a short description of what happened.	
Root Causes: Include a description of any conditions that contributed to the issue. If there were any actions taken that exacerbated the issue, also include them here with the attention of learning from any mistakes made during the resolution process.	
Resolution: Include a description of what caused the problem. If there was a temporary fix in place, describe that along with the long term solution.	

pagerduty

Un post mortem :

- Summary
- Root Cause
- The impact
- What happened
- Exact Timeline
- Lesson learned (what went well, what went bad)
- Actions to prevent the same incident

Guide de post mortem

Un SRE pour résumer :

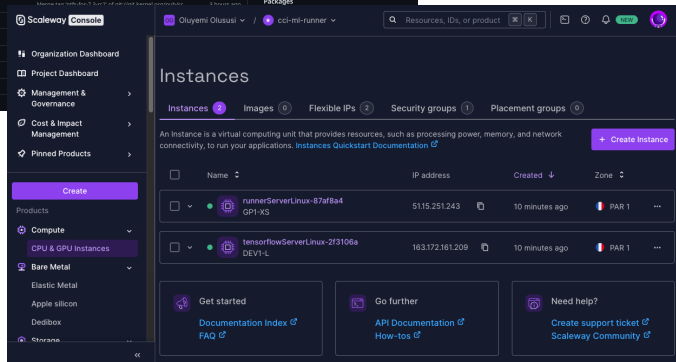
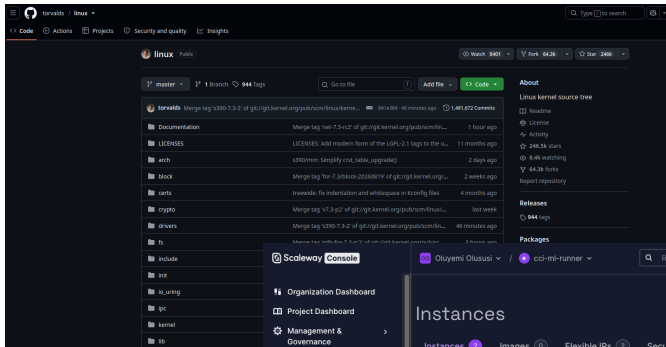
- Responsable de la production
- Assure la stabilité et la fiabilité
- Utilise les méthodes d'ingénierie informatique pour la production
- Beaucoup de métriques et d'automatisations
- Une maîtrise de la performance et de la sécurité de la production

La seule question qu'il se pose :

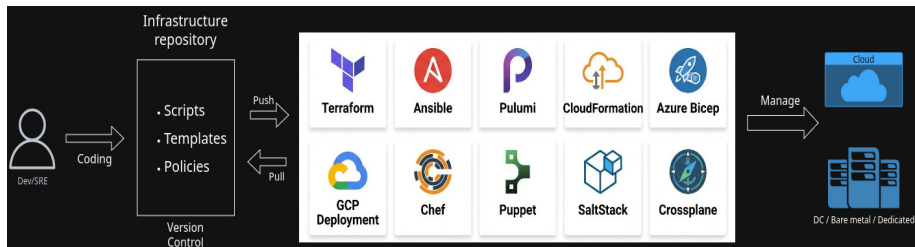
- Comment mieux faire de la production au rythme des développeurs ?

Questions ?

Concrètement, on fait quoi en étant que SRE ?



IAC :

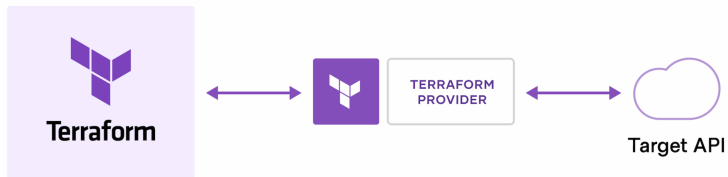


Quelques propriétés importantes :

- *"Version control" + "Automation"*
- Déclaratif vs Impératif
- Tests automatiques
- Plus de modularité (factorisation)
- Moins de *"configuration drift"*
- *"Cloud native"*

Quelques challenges/désavantages :

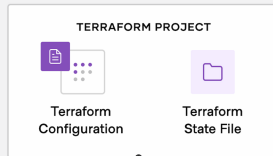
- Complexité des pipelines/automatisations
- Sécurité standardisée
- Migration des infrastructures *legacy*
- L'échelle des opérations



On écrit du Terraform afin d'utiliser un provider qui sert à parler à une API

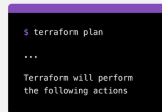
Write

Define infrastructure in configuration files



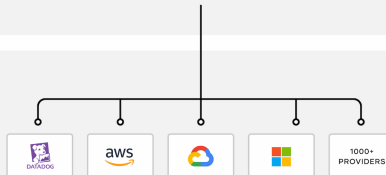
Plan

Review the changes
Terraform will make to
your infrastructure



Apply

Terraform provisions
your infrastructure and
updates the state file.



Terraform :

- Écrit en Go
- Évolutif (les providers sont versionnés)
- Gère les dépendances
- Système de workspace
- Stateful
- Un catalog de providers avec leur documentation

 Terraform | Registry

Search providers, modules, run tasks, and policies...

Providers

Modules

Run Tasks

Policies

Filters

x Clear Filters

Tier

☒ Official

Official providers are owned and maintained by HashiCorp

☒ Partner

Partner providers are owned and maintained by a technology company that has gone through the partner onboarding process and maintain a direct partnership with HashiCorp

☒ Partner Premier

Technology partners are third-party companies that write and maintain partner premier providers. To earn a partner premier badge, the partner must qualify

☒ Community

Community providers are published and maintained by individual contributors of the ecosystem

Category

☐ Asset Management

☐ Cloud Automation

☐ Communication & Messaging

☐ Container Orchestration

☐ Continuous Integration/Deployment (CI/CD)

☐ Data Management

☐ Database

☐ Infrastructure (IaaS)

☐ Logging & Monitoring

☐ Networking

☐ Platform (PaaS)

☐ Security & Authentication

Providers

Providers are a logical abstraction of an upstream API. They are responsible for understanding API interactions and exposing resources.

Featured Providers

 AWS
by HashiCorp
Docs

 Azure
by HashiCorp
Docs

 Google Cloud Platform
by HashiCorp
Docs

 Kubernetes
by HashiCorp
Docs

 Alibaba Cloud
by aliyun
Docs

 Oracle Cloud Infrastructure
by oracle
Docs

Official Providers

 Aap
by ansible
Docs

 Active Directory
by HashiCorp
Docs

 Archive
by HashiCorp
Docs

 AWS Cloud Control
by HashiCorp
Docs

 Azure Active Directory
by HashiCorp
Docs

 Azure Stack
by HashiCorp
Docs

 Boundary
by HashiCorp
Docs

 Cloudinit
by HashiCorp
Docs

 Consul
by HashiCorp
Docs

Maxime El Haddari

Culture et fondations DevOps/SRE

September 16, 2026

40 / 77

On configure le provider AWS

```
provider "aws" {
```

```
    region = "us-east-1"
```

```
}
```

Appel de la ressource aws_instance connue du provider aws

```
resource "aws_instance" "example" {
```

```
    ami = data.aws_ami.ubuntu.id
```

```
    instance_type = "t3.micro"
```

```
    tags = {
```

```
        Name = "HelloWorld"
```

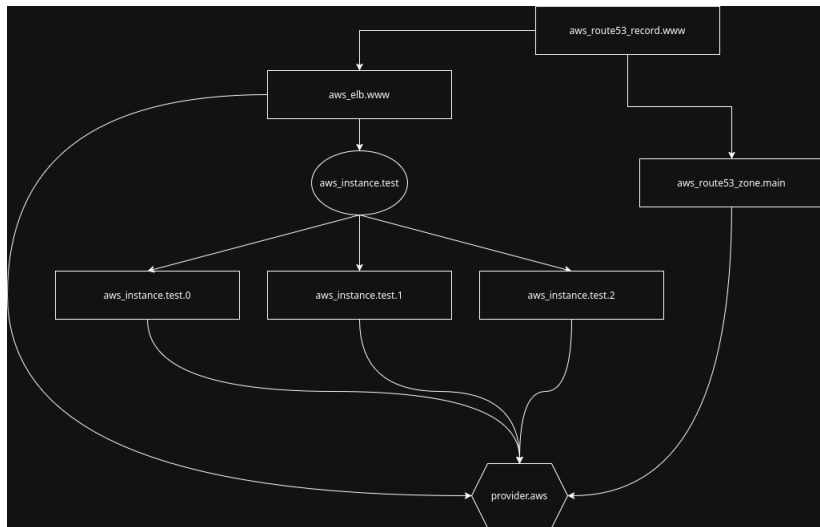
```
    }
```

```
}
```

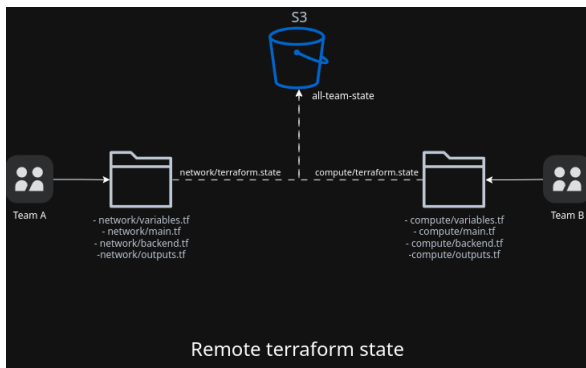
On récupère une information déjà existante

```
data "aws_ami" "ubuntu" {
  most_recent = true
  filter {
    name      = "name"
    values    = ["ubuntu/*/ubuntu-jammy-*-amd64-server-*"]
  }
  filter {
    name      = "virtualization-type"
    values    = ["hvm"]
  }
  owners = ["099720109477"] # Canonical
}
```

Exemple d'architecture :



Partager le state :



Partager le state :

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "4.24.0"  
    }  
  }  
}  
  
# On définit un backend (par défaut le state est local)  
backend "s3" {  
  bucket      = "tf-bucket-ensiie"  
  key         = "all-team-state.tfstate"  
  region      = "eu-west-2"  
  profile     = "admin_profile"  
}  
}
```

<https://github.com/terraform-aws-modules/terraform-aws-ec2-instance/>

```
module "ec2_instance" {  
    source = "terraform-aws-modules/ec2-instance/aws"  
    name = "single-instance"  
    instance_type = "t3.micro"  
    key_name = "user1"  
    monitoring = true  
    subnet_id = "subnet-eddczz4"  
    tags = {  
        Terraform = "true"  
        Environment = "dev"  
    }  
}
```

<https://github.com/terraform-aws-modules/terraform-aws-ec2-instance/>

```
resource "aws_eip" "this" {  
  ...  
}  
resource "aws_vpc_security_group_ingress_rule" "this" {  
  ...  
}  
resource "aws_ebs_volume" "this" {  
  ...  
}  
resource "aws_volume_attachment" "this" {  
  ...  
}
```

<https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks>

```
dynamic "network_interface" {  
  for_each = var.network_interface != null ? var.network_interface : {}  
  content {  
    delete_on_termination = network_interface.value.delete_on_termination  
    device_index = coalesce(network_interface.value.device_index, network_interface.key)  
    network_card_index = network_interface.value.network_card_index  
    network_interface_id = network_interface.value.network_interface_id  
  }  
}
```

```
└─ layers
  └─ bootstrap
    ├── _settings.tf
    └── bootstrap.tf
  └─ core
    ├── _settings.tf
    ├── common.tf
    ├── keyvault.tf
    ├── landing_zone.tf
    ├── monitoring.tf
  └─ dev
    ├── _settings.tf
    └── application.tf
└─ modules / starter-managed
  ├── acr.tf
  ├── app_gateway.tf
  ├── application.tf
  ├── database.tf
  ├── monitoring.tf
  └── variables.tf
```

🌟 Standards 🌟

The 3 gold standards for your infrastructure codebase

- Your layer design reflects a business need
- Only one team contributes has ownership of a layer (you have a clear vision of contribution & ownership)
- Your state refresh time is acceptable for a contribution purpose

The standards for layers

- You can rebuild your infrastructure without the `-target` flag
- You do not have any cyclical dependencies between resources
- You use data blocks only in layers

The standards for modules

- A module should be created
 - To abstract complexity
 - If you have to reuse it frequently (More than twice)
- You should not create a module to be a flat pass between resources
- Your module's dependencies are provided by the caller
- You use remote modules to avoid repetition between your different projects

The standard naming convention

- The name of the file or folder should reflect what you are creating
- Adapt your codebase to your client needs and standards
- Avoid stuttering when naming Terraform resources.
- Use snake_case convention

- Standards for layers :
 - ▶ You can rebuild your stack without `-target` flag
 - ▶ You do not have any cyclical dependencies between resources
 - ▶ You use data blocks only in layers
- Standards for modules :
 - ▶ A module should be created to abstract complexity or heavily reuse the module
 - ▶ You should not create a module to be a flat pass between resources
 - ▶ Your module's dependencies are provided by the caller
 - ▶ You use remote modules to avoid repetition between your different projects

Comment on peut facilement travailler sur la même infrastructure Terraform ?



- On crée une Pull Request
- On peut effectuer le plan depuis la PR
- Quand quelqu'un valide la PR, on peut apply
- Tout reste dans le repo

Layers

Search into layers

144 layers | Filter by State Repository Hide Pull Requests

Layer Name	State	Repository	Branch	Path	Last result
shared-iam	Running	burrito-sentry	shared/iam	main	
dev-api-gateway	OK	burrito/blueprint	main	layers/api-gateway/dev	Plan: 0 to create, 0 to update, 0 to delete
dev-backend	OutOfSync	burrito/blueprint	main	layers/backend/dev	Plan: 0 to create, 1 to update, 0 to delete
dev-datasources	OK	burrito/blueprint	main	layers/datasources/dev	Plan: 0 to create, 0 to update, 0 to delete
prod-api-gateway	OK	burrito/blueprint	main	layers/api-gateway/prod	Plan: 0 to create, 0 to update, 0 to delete
prod-datasources	OK	burrito/blueprint	main	layers/datasources/prod	Plan: 0 to create, 0 to update, 0 to delete
prod-frontend	OK	burrito/blueprint	main	layers/frontend/prod	Plan: 0 to create, 0 to update, 0 to delete

burrito

License Apache 2.0 | go report | build | cdktf | 100% | on Chat | Discard



Burrito is a TACO5 (Terraform Automation Collaboration Software) Kubernetes Operator.

Why does this exist?

[Terraform](#) is a tremendous tool to manage your infrastructure in IaC. However, it lacks built-in solutions for managing [state drift](#).

Additionally, configuring a CI/CD pipeline for Terraform can be challenging and often varies depending on the selected tools.

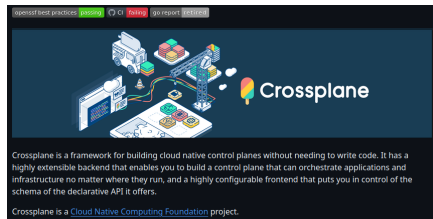
Finally, currently, there is no easy way to navigate your Terraform state to truly understand the modifications it undergoes when running `terraform apply`.

[burrito](#) aims to tackle those issues by:

- Planning continuously your Terraform code and running applies if needed
- Offering an out-of-the-box PR/MR integration so you do not have to write CI/CD pipelines for Terraform ever again
- Showing your state's modifications in a simple Web UI



Terraform fork, highly compatible

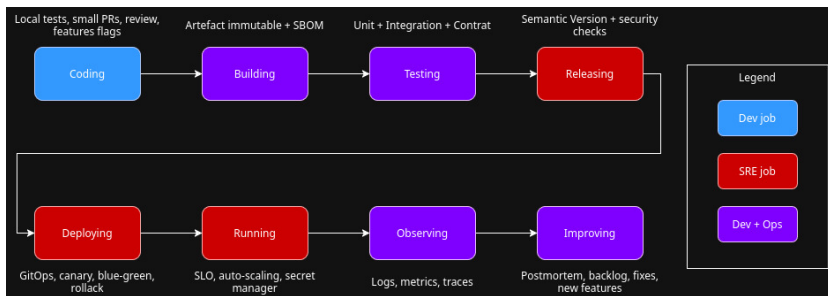


Creates custom APIs in Kubernetes
to manage your Infra and
applications

Questions ?

On sait faire un peu de TF, on est capable de créer son infra mais finalement comment on déploie son code dessus ?

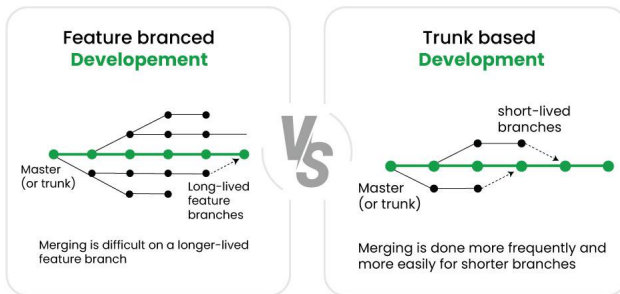
SDLC : qu'est-ce que ça veut dire ?



Coding :

- Trunk-based (branches courtes)
- Easy review, easy life
- Add feature flags
- Quelques conventions
- lint + unit tests + pre-commit

Trunk-based Development :



- On réduit l'effort pour merge
- La branche main représente du code valide et testé donc go en prod
- On ajoute beaucoup de code petit à petit (features flags)

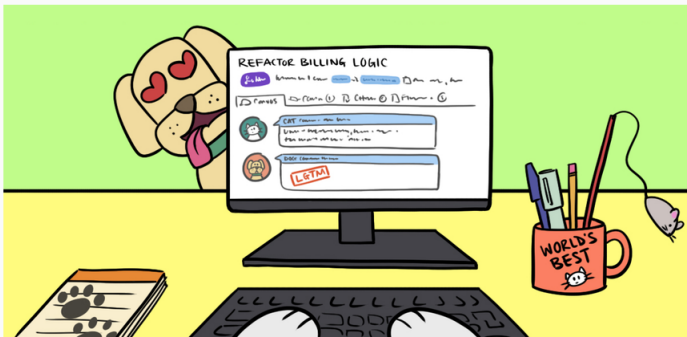
Les reviews :

<https://mtlynch.io/code-review-love/>



Michael Lynch

[Blog](#) [Retrospectives](#) [Projects](#) [Book Reports](#) [Notes](#) [About](#)



How to Make Your Code Reviewer Fall in Love with You

📅 December 2, 2020 ⌚ 14-minute read 🏷️ [code review](#) • [culture](#) • [code style](#)

Les reviews :

- Review your code first

Les reviews :

- Review your code first
- Write a clear changelist description

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques
- Be patient when your reviewer is wrong

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques
- Be patient when your reviewer is wrong
- Communicate your responses explicitly

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques
- Be patient when your reviewer is wrong
- Communicate your responses explicitly
- Artfully solicit missing information

Les reviews :

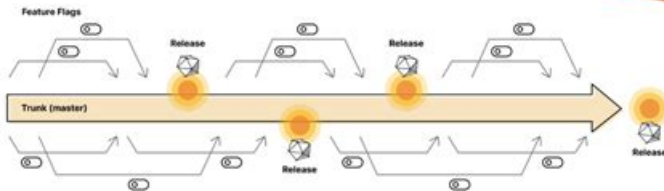
- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques
- Be patient when your reviewer is wrong
- Communicate your responses explicitly
- Artfully solicit missing information
- Award all ties to your reviewer

Les reviews :

- Review your code first
- Write a clear changelist description
- Automate the easy stuff (pre-commit, linter, etc ...)
- Answer questions with code
- Narrowly scope changes
- Separate functional and non-functional changes
- Break up large changelists
- Respond with grace to critiques
- Be patient when your reviewer is wrong
- Communicate your responses explicitly
- Artfully solicit missing information
- Award all ties to your reviewer
- Minimize lag between rounds of review

Les Feature Flags :

Trunk-Based Development with Feature Flags



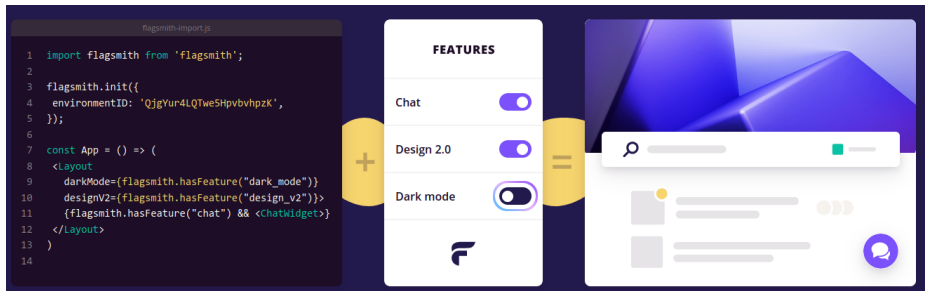
Les Feature Flags :



- On gère le déploiement d'une feature sans redéployer
- La feature est disponible pour un groupe d'utilisateurs
- On maîtrise le déploiement de la feature progressivement

Les problématiques des Feature Flags:

- Débugger le parcours d'un utilisateur peut devenir plus compliqué
- On maintient beaucoup plus de code (potentiellement mort)
- On a beaucoup plus d'edge cases à gérer



- Beaucoup de FF système disponible (avec intégration dans beaucoup de langages)
- Permet de gérer les FF de manière centralisée et à distance

<https://www.conventionalcommits.org/en/v1.0.0/>

Conventional Commits

A specification for adding human and machine readable meaning to commit messages

Quick Summary

Full Specification

Contribute



The commit contains the following structural elements, to communicate intent to the consumers of your library:

1. **fix:** a commit of the type `fix` patches a bug in your codebase (this correlates with `PATCH` in Semantic Versioning).
2. **feat:** a commit of the type `feat` introduces a new feature to the codebase (this correlates with `MINOR` in Semantic Versioning).
3. **BREAKING CHANGE:** a commit that has a footer `BREAKING CHANGE:` , or appends a `!` after the type/scope, introduces a breaking API change (correlating with `MAJOR` in Semantic Versioning). A BREAKING CHANGE can be part of commits of any type.
4. types other than `fix:` and `feat:` are allowed, for example `@commitlint/config-conventional` (based on the `Angular convention`) recommends `build:` , `chore:` , `ci:` , `docs:` , `style:` , `refactor:` , `perf:` , `test:` , and others.
5. footers other than `BREAKING CHANGE: <description>` may be provided and follow a convention similar to `git trailer format`.

Additional types are not mandated by the Conventional Commits specification, and have no implicit effect in Semantic Versioning (unless they include a BREAKING CHANGE). A scope may be provided to a commit's type, to provide additional contextual information and is contained within parenthesis, e.g., `feat(parser): add ability to parse arrays` .

Examples

Commit message with description and breaking change footer

```
feat: allow provided config object to extend other configs  
  
BREAKING CHANGE: `extends` key in config file is now used for extending other config files
```

Commit message with **!** to draw attention to breaking change

```
feat!: send an email to the customer when a product is shipped
```

Commit message with scope and **!** to draw attention to breaking change

```
feat(api)!: send an email to the customer when a product is shipped
```

Commit message with both **!** and BREAKING CHANGE footer

```
feat!: drop support for Node 6  
  
BREAKING CHANGE: use JavaScript features not available in Node 6.
```

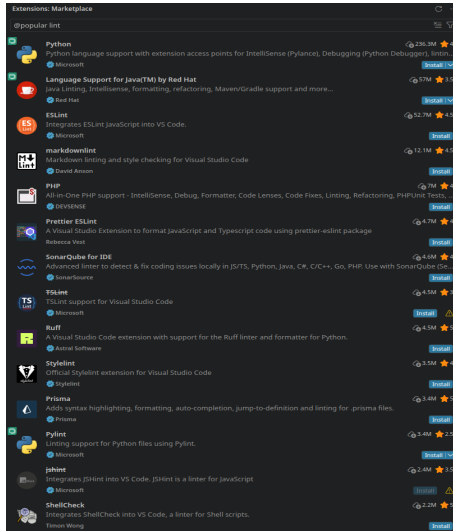
Commit message with no body

```
docs: correct spelling of CHANGELOG
```

Pourquoi ?

- Rend la review plus simple pour un humain (le commit est la plus petite unité de sens de la review)
- Générer un CHANGELOG automatiquement
- Donne une vision plus claire à vos utilisateurs/développeurs
- Le versioning devient plus simple à automatiser (SemVer)

lint + tests + pre-commit = easy review



Pre-commits + lint + tests = easy review

Navigate to the .git/hooks directory in your repository

```
$ cd .git/hooks
```

Create the pre-commit script

```
$ touch pre-commit
```

Make the script executable

```
$ chmod +x pre-commit
```

Edit the pre-commit script

```
$ nano pre-commit
```

Add the following script

```
#!/bin/sh
```

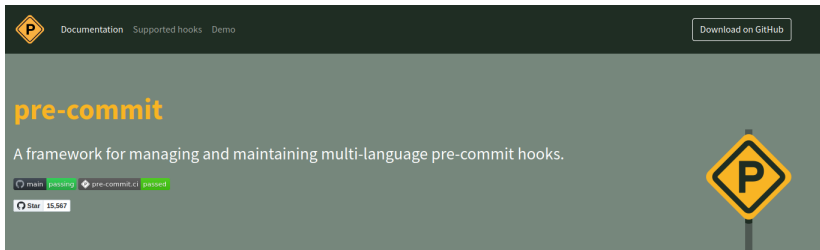
```
if git grep --cached -q 'TODO'; then
```

```
    echo 'TODO comments found. Resolve before committing.'
```

```
    exit 1
```

```
fi
```

Pre-commits + lint + tests = easy review

[Introduction](#)[Installation](#)[Adding plugins](#)[Usage](#)[Creating new hooks](#)[Command line interface](#)[Advanced features](#)[Contributing](#)

Introduction

Git hook scripts are useful for identifying simple issues before submission to code review. We run our hooks on every commit to automatically point out issues in code such as missing semicolons, trailing whitespace, and debug statements. By pointing these issues out before code review, this allows a code reviewer to focus on the architecture of a change while not wasting time with trivial style nitpicks.

As we created more libraries and projects we recognized that sharing our pre-commit hooks across projects is painful. We copied and pasted unwieldy bash scripts from project to project and had to manually change the hooks to work for different project structures.

We believe that you should always use the best industry standard linters. Some of the best linters are written in languages that you do not use in your project or have installed on your machine. For example scss-lint is a linter for SCSS written in Ruby. If you're writing a project in node you should be able to use scss-lint as a pre-commit hook without adding a Gemfile to your project or understanding how to get scss-lint installed.

We built pre-commit to solve our hook issues. It is a multi-language package manager for pre-commit hooks. You specify a list of hooks you want and pre-commit manages the installation and execution of any hook written in any language before every commit. pre-commit is specifically designed to not require root access. If one of your developers doesn't have node installed but modifies a JavaScript file, pre-commit automatically handles downloading and building node to run eslint without root.

Comment on déploie notre code ?

La suite au prochain épisode

Les enjeux :

- 24/7 disponibilité dans le monde
- Scalabilité de l'archi et des process/outils
- Réduction du time from code to user
- Sécurité by design (pas à la fin)
- Utilisation des métriques partout (prise de décisions)

Les principes :

- Tout automatiser surtout le toil
- Échouer avec grâce (on casse staging avant la prod)
- CI/CD on déploie tous les jours
- Infra as Code
- Documentation et team play
- Fiabilité & stabilité > sauveur de prod

Les pratiques:

- IaC: Terraform, modules, state
- SDLC: code -> build -> test -> release -> run -> observe
- How to code: trunk-based, quick & easy review, use feature flags, respect some common convention, lint + test + pre-commits

Les notions clefs :

- SLI / SLO / SLA: indicateurs / objectifs / engagements
- Error budget: marge d'indispo, impacte le rythme de déploiements
- Toil: travail répétitif sans valeur
- Réponse à incident: rôles, post-mortem, actions préventives

Exemples SLO :

- 99.9 SLO / 30 jours = 43,2 minutes d'indispo
- 20 minutes d'indisponibilité = 46% du budget cramé
donc on ralentit les déploiements de features
- Burn-rate est intéressant à suivre également

Questions ?

Quelques sources :

- [Atl] Atlantis. *Atlantis : GitHub repository*. URL: <https://github.com/runatlantis/atlantis>. (accessed: September 3, 2026).
- [Cla] Fabrice Claeys. *Histoire et Origines du Mouvement DevOps*. URL: <https://foreach-academy-france.github.io/formation-devops-initiation/docs-html/01-histoire-devops.html>. accessed: September 3, 2026.
- [Com] Conventional Commits. *Conventional Commits*. URL: <https://www.conventionalcommits.org/en/v1.0.0/>. (accessed: September 3, 2026).
- [Goo] Google. *Site Reliability Engineering*. URL: <https://sre.google/workbook/table-of-contents/>. (accessed: September 2, 2026).

- [Lyn] Michael Lynch. *How to Make Your Code Reviewer Fall in Love with You*. URL: <https://mtlynch.io/code-review-love/>. (accessed: September 3, 2026).
- [Red] Redhat. *What is Infrastructure as Code (IaC)?* URL: <https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code-iac>. (accessed: September 3, 2026).
- [Sot] Anthony Sottile. *Pre-commits*. URL: <https://pre-commit.com/>. (accessed: September 3, 2026).